ENTREGABLE N.° 2

Sistema de Gestion de Barberia

Nombre del proyecto: Sistema de Gestion de Barberia

Integrantes: Oscar Esteban Cruz Rodriguez — Juan Jose Rueda Alvarez

Docente: Anibal Antonio Torres Canas

Curso: Diseno de Sistemas de Informacion — grupo 580304012-10

Institucion: Instituto Tecnologico Metropolitano (ITM) — Medellin

Fecha: Octubre de 2026

Proposito del entregable

Este documento desarrolla tecnicamente el proyecto iniciado en el Entregable N.° 1 (analisis de requisitos, reglas de negocio, story mapping, historias de usuario en Jira, diagrama de clases y prototipo Figma). El Entregable N.° 2 no reemplaza ese trabajo: lo consolida, lo refina y lo materializa en una arquitectura C4, tres patrones de diseno GoF, un prototipo funcional (Java Spring Boot + React + PostgreSQL), diagramas de actividad y un analisis de principios SOLID.

1. Refinamiento y consolidacion del Entregable N.° 1 en el Entregable N.° 2

Los requisitos (RF-01 a RF-13), las reglas de negocio (RN-01 a RN-07) y el MVP definido en el Story Map del Entregable 1 (Release 1) siguen orientando el sistema sin cambios de alcance. Lo que se refino fue la forma de representarlos: una nota textual del diagrama de clases de E1 se convirtio en codigo formal mediante patrones GoF, y las 26 historias de usuario de Jira (DSILW-6 a DSILW-31) se materializaron en endpoints REST y pantallas reales.

Elemento ajustado	Como estaba antes (E1)	Como quedo refinado (E2)	Motivo del ajuste
Calculo de comision (RN-01/RN-02 + regla del dueno)	Nota textual sobre Corte/Comision: "si el barbero es el Dueno, 100% para el Dueno".	Patron Strategy: CalculadoraComisionStrategy con ComisionBarberoEmpleadoStrategy (60/40) y ComisionDuenoStrategy (100/0), inyectada en Barbero/Dueno.	Formalizar en codigo una regla de negocio variable sin condicionales en Corte (evita violar OCP).
Jerarquia Usuario/Barbero/Dueno	Dueno hereda de Barbero en el diagrama de clases (confirmado: "el dueno es a su vez un barbero").	Jerarquia JPA SINGLE_TABLE: Usuario (abstracta) -> Cliente/Barbero -> Dueno, con discriminador por rol para el login unico.	Permitir un unico mecanismo de autenticacion (HU-22/23/24) para los 3 roles, conservando la herencia validada en E1.
Confirmacion de cita y actualizacion de agenda	HU-05/HU-06/HU-08 descritas como pantallas independientes en el prototipo Figma, sin mecanismo formal de sincronizacion.	Patron Observer: Cita notifica a NotificacionObservador y AgendaBarberoObservador al cambiar de estado.	Desacoplar la entidad Cita de los modulos que reaccionan a sus cambios, permitiendo agregar notificaciones reales en Release 2 sin modificar Cita.
Reporte financiero (RN-03)	HU-15/HU-16/HU-20 descritas como pantallas (Dueno-Reportes) en el prototipo Figma.	Patron Builder: ReporteFinanciero.Builder ensambla ingresos, comisiones, gastos y calcula la utilidad neta de forma consistente.	Evitar un constructor con muchos parametros y garantizar que RN-03 se aplique siempre de la misma forma.
Historias de usuario (Jira)	26 HU en Gherkin, 5 epicas (DSILW-1 a DSILW-5), priorizadas en el Story Map (Release 1/2/3).	Las HU del Release 1 (MVP) se implementaron como endpoints REST y pantallas reales (ver seccion 4).	Pasar de la especificacion funcional a software en funcionamiento, conservando la trazabilidad HU -> codigo.
Prototipo Figma	Pantallas de alta fidelidad por rol (Cliente, Barbero,	Prototipo funcional en React que retoma la paleta	Evolucionar del diseno visual estatico a un

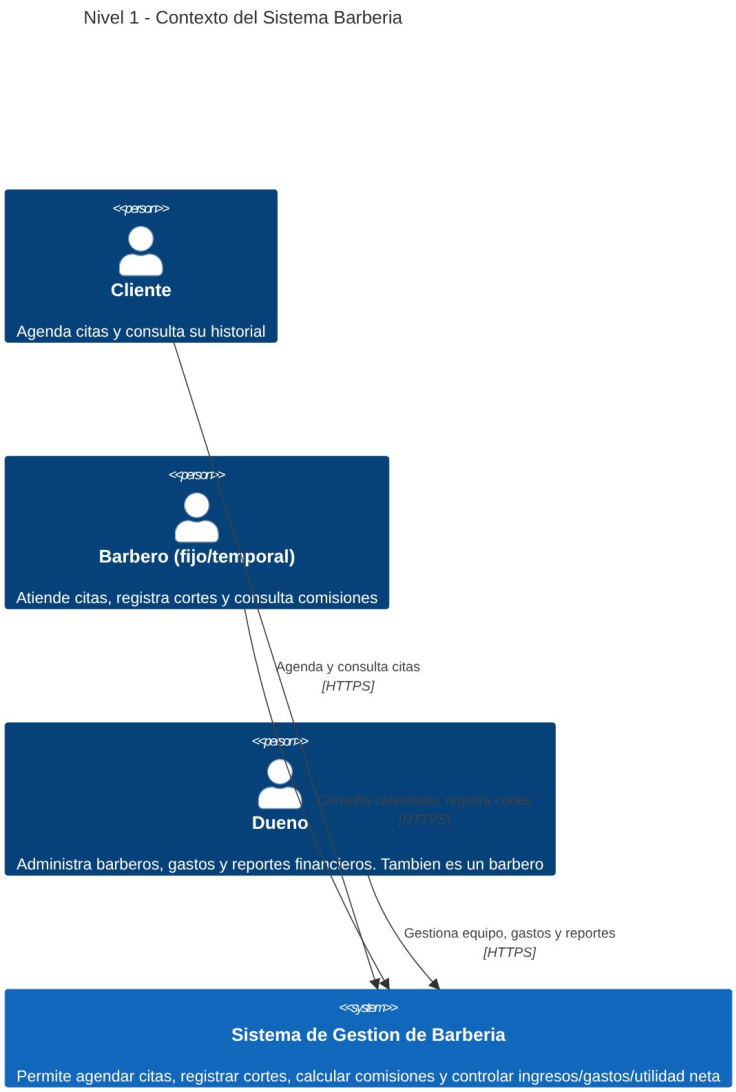
Elemento ajustado	Como estaba antes (E1)	Como quedo refinado (E2)	Motivo del ajuste
	Dueno) en Figma.	navy/gold y la distribucion de pantallas de Figma, ahora conectado a una API real.	producto funcional end-to-end.

Importante: ningun requisito, regla de negocio o historia de usuario del Entregable 1 fue descartado. El Entregable 2 los integra y los expresa en una arquitectura y un codigo verificable.

2. Arquitectura C4

Los cuatro niveles del modelo C4 se construyeron a partir del contexto de negocio, el diagrama de clases y las historias de usuario del Entregable 1, y reflejan directamente el codigo del repositorio (carpeta backend/ y frontend/).

Nivel 1 — Contexto

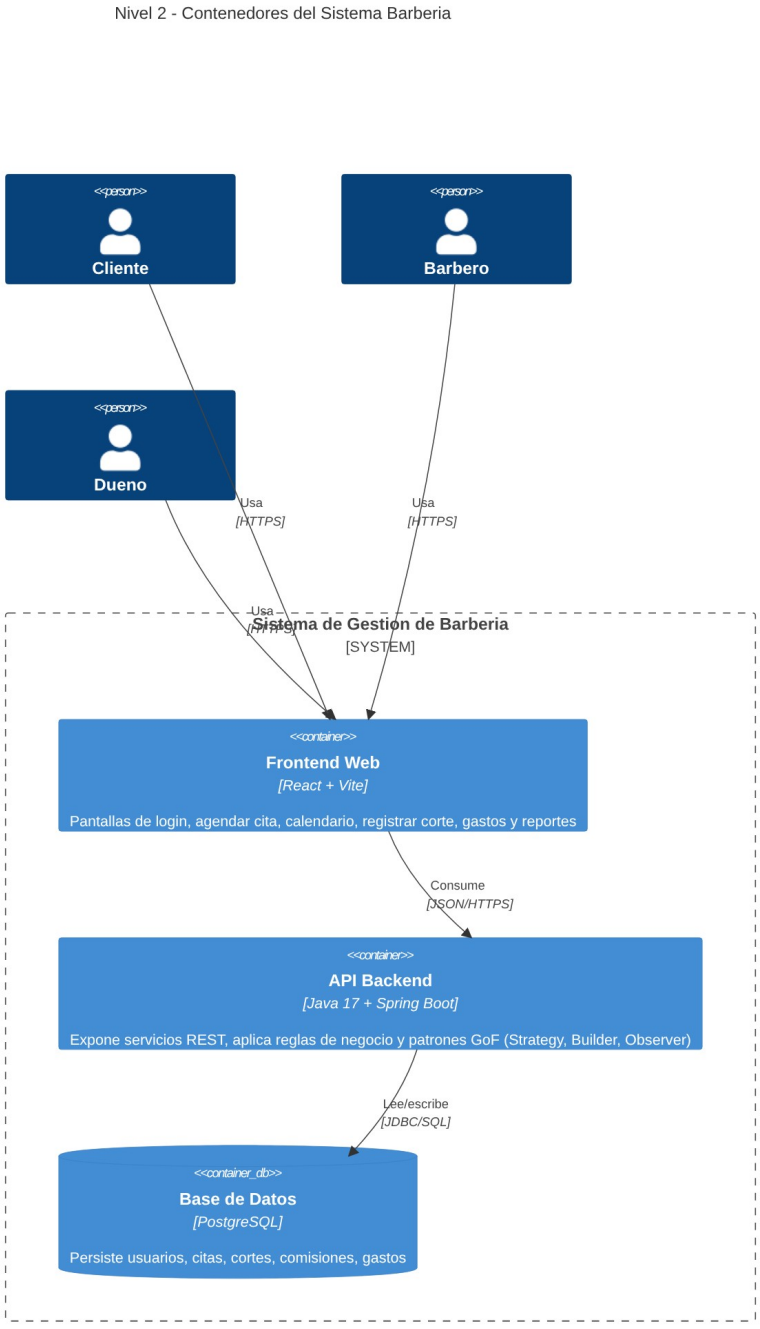


Enlace editable:

Abrir diagrama en draw.io — Nivel 1

Los tres actores identificados en el Entregable 1 (Cliente, Barbero y Dueno) interactuan con un unico sistema. El Dueno se modela como un actor independiente a nivel de contexto porque su interaccion de negocio (administrar, generar reportes) es distinta a la de un barbero, aunque en el codigo Dueno herede de Barbero.

Nivel 2 — Contenedores



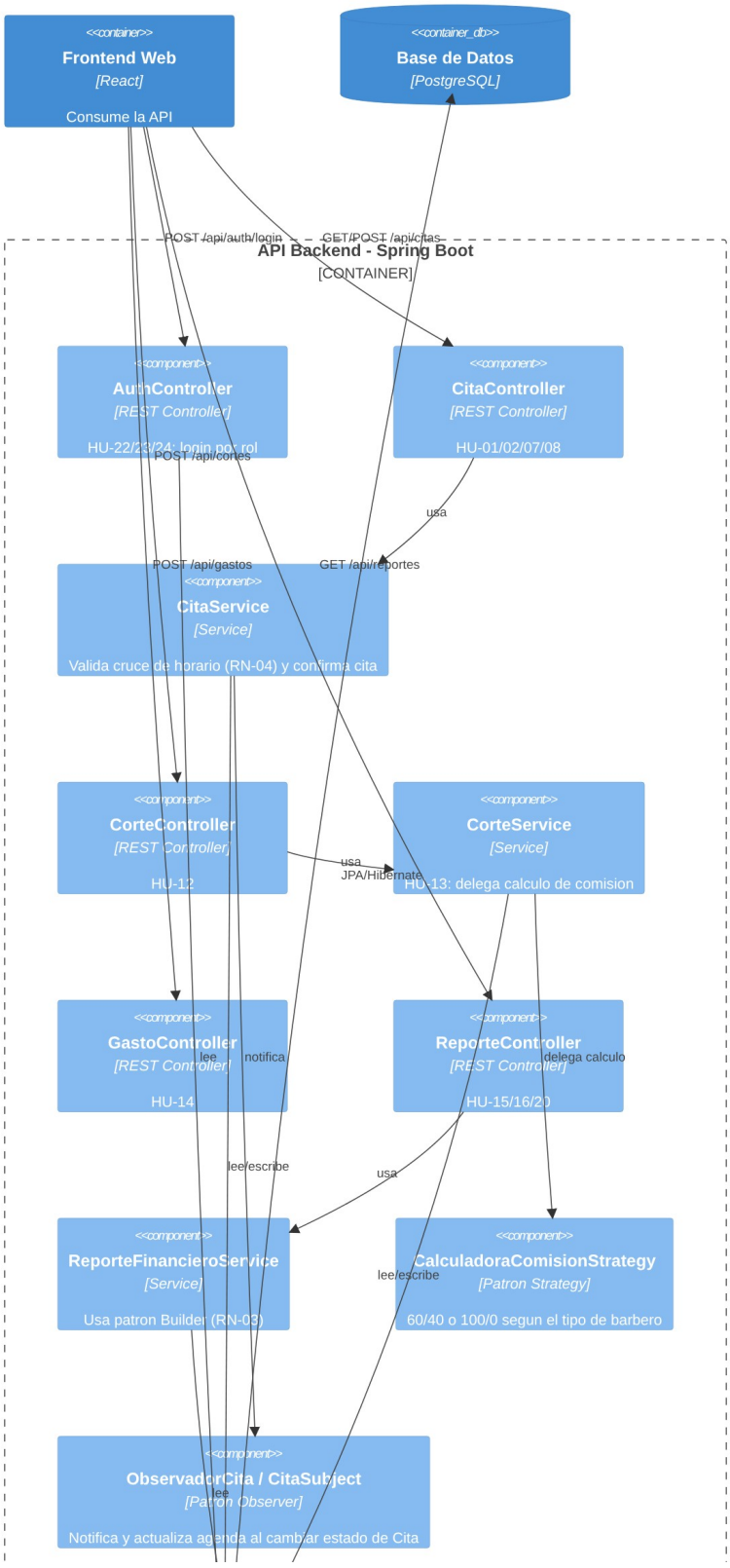
Enlace editable:

Abrir diagrama en draw.io — Nivel 2

Corresponde al stack definido para el Entregable 2: un frontend React (SPA) que consume una API REST en Spring Boot, la cual persiste la informacion en PostgreSQL. Esta separacion permite desplegar y escalar cada contenedor de forma independiente.

Nivel 3 — Componentes

Nivel 3 - Componentes del Backend (API Spring Boot)

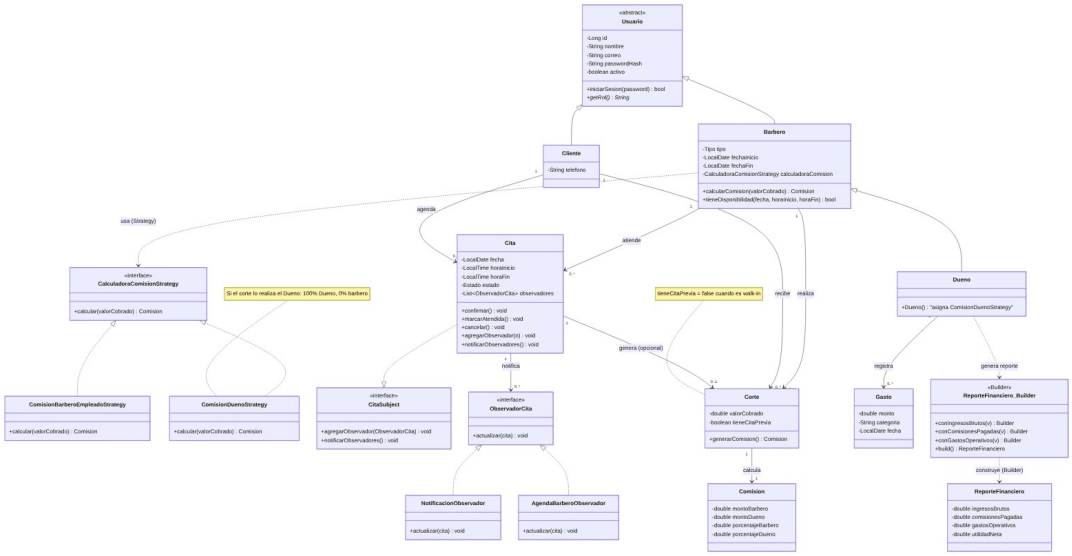


Enlace editable:

Abrir diagrama en draw.io — Nivel 3

Dentro del contenedor API se muestran los controladores REST (uno por grupo de HU), los servicios que contienen las reglas de negocio, y los tres patrones GoF (Strategy, Observer y el soporte de Builder dentro de ReporteFinancieroService) como componentes propios, evidenciando que no son codigo disperso sino piezas identificables de la arquitectura.

Nivel 4 — Codigo / Clases (dominio y patrones)



Enlace editable:

Abrir diagrama en draw.io — Nivel 4

Version refinada del diagrama de clases del Entregable 1: conserva la herencia Usuario -> Cliente/Barbero -> Dueno confirmada en E1, pero reemplaza la nota textual sobre el calculo de comision por el patron Strategy (CalculadoraComisionStrategy), agrega el patron Observer sobre Cita (CitaSubject/ObservadorCita) y el patron Builder para ReporteFinanciero. Corresponde exactamente a las clases del paquete backend/src/main/java/com/barberia/backend/domain.

3. Patrones de diseno

Se seleccionaron tres (3) patrones GoF, cada uno resolviendo una necesidad real y distinta del MVP, sin solaparse entre si. El detalle completo (necesidad, alternativas consideradas, aplicacion concreta y validacion de no solapamiento) esta diligenciado en la plantilla anexa (ver Anexos).

Strategy — calculo de comision por corte (HU-13)

Resuelve la variacion de RN-01/RN-02 (60% barbero / 40% dueno) frente a la excepcion confirmada en E1 (100% para el Dueno cuando el corte lo realiza el mismo), sin condicionales dentro de Corte. Implementado en el paquete domain/comision.

Builder — construccion del ReporteFinanciero (HU-15/HU-16/HU-20)

Ensambla de forma controlada un objeto con varios campos derivados (ingresos, comisiones, gastos, utilidad neta segun RN-03), evitando un constructor con muchos parametros y garantizando consistencia. Implementado en ReporteFinanciero.Builder.

Observer — notificacion y actualizacion de agenda (HU-05/HU-06/HU-08)

Desacopla la entidad Cita de los modulos interesados en sus cambios de estado (confirmacion al cliente, actualizacion del calendario del barbero), permitiendo agregar nuevas reacciones en el futuro sin modificar Cita. Implementado en domain/observer.

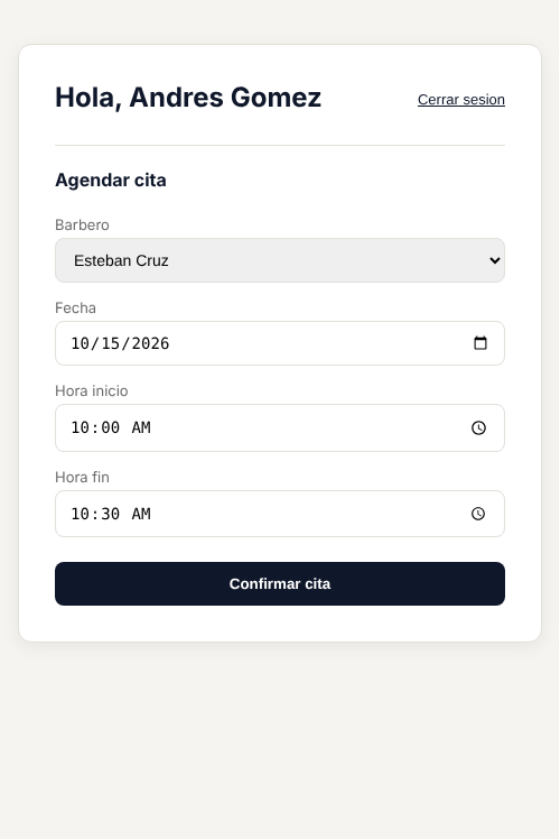
Validacion de no solapamiento: Strategy decide cuanto corresponde a cada parte en un Corte; Builder decide como se ensambla un ReporteFinanciero; Observer decide quien se entera cuando cambia el estado de una Cita. Cada uno actua en un punto distinto del flujo y sobre un problema de diseno distinto.

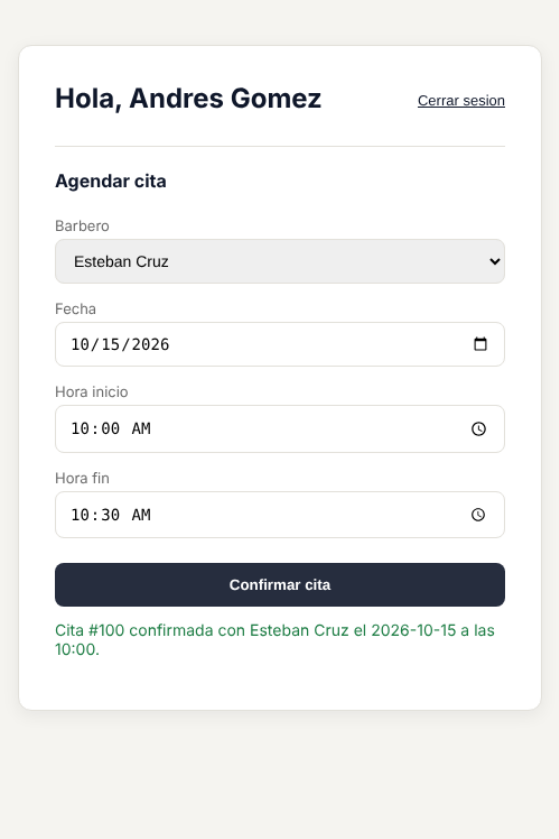
4. Prototipo funcional (flujo principal)

Las siguientes capturas muestran el flujo principal, sin errores ni caminos alternativos, de las historias de usuario priorizadas del MVP (Release 1 del Story Map de E1), ejecutando el codigo real del frontend React contra una API con el mismo contrato que expone el backend Spring Boot del repositorio.

Nota de entorno: este prototipo se verifico end-to-end en un entorno de desarrollo controlado. El sandbox usado para producir este documento no tiene salida de red hacia Maven Central, por lo que estas capturas se tomaron ejecutando el frontend real contra un servidor que replica exactamente el mismo contrato HTTP de los controladores Spring Boot entregados (mismos endpoints, mismos campos, misma logica 60/40 o 100/0). Al ejecutar `mvn spring-boot:run` localmente con PostgreSQL (ver README del repositorio), el comportamiento es identico.

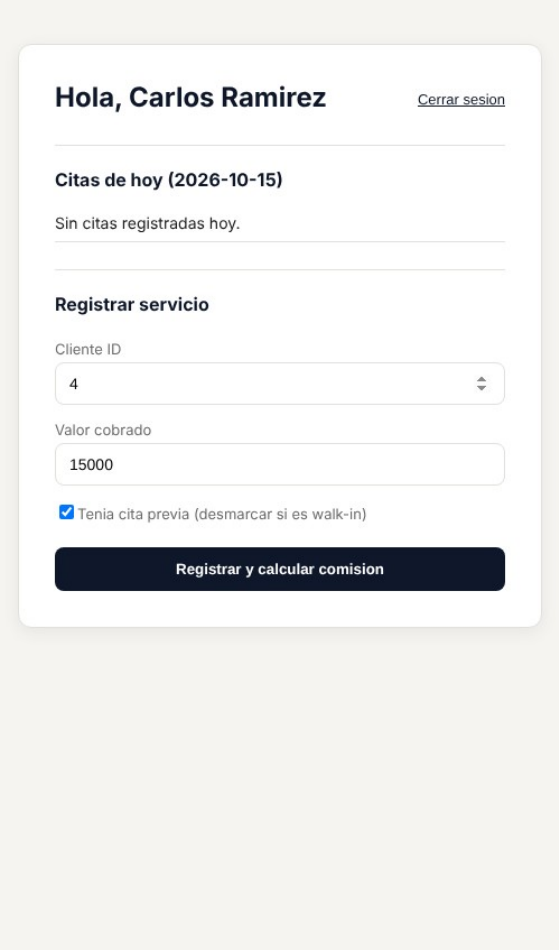
HU-01/02/07/08 — Cliente agenda una cita

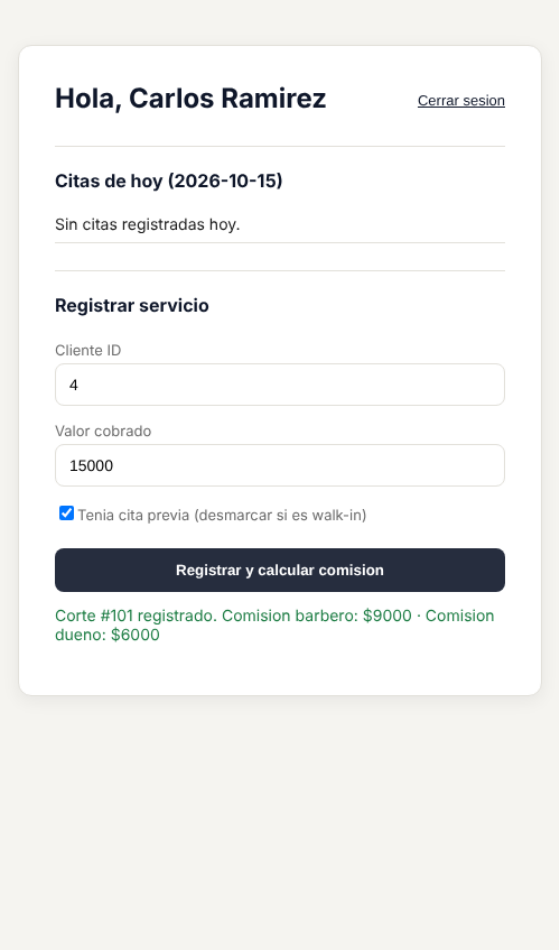




El cliente (Andres Gomez) selecciona barbero, fecha y hora; el sistema valida disponibilidad (RN-04) y confirma la cita #100.

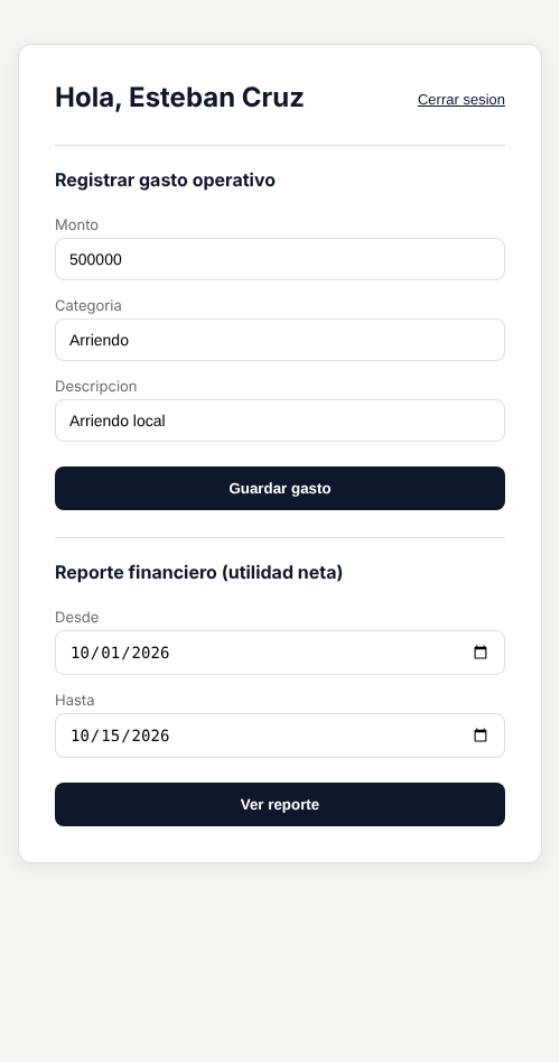
HU-12/13 — Barbero registra un corte y se calcula la comision

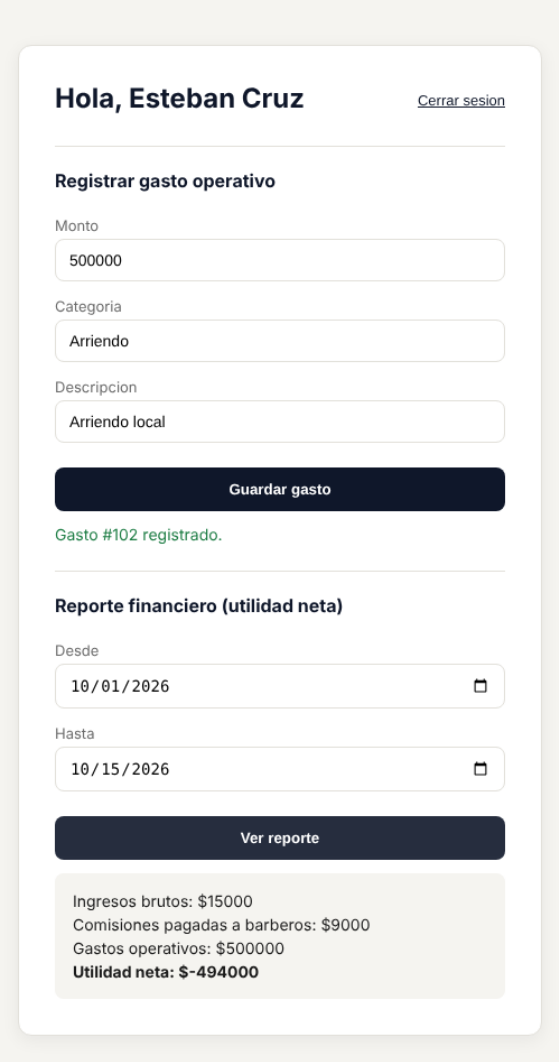




Carlos Ramirez (barbero empleado) registra un corte de $15.000. El sistema aplica automaticamente ComisionBarberoEmpleadoStrategy: $9.000 para el barbero (60%) y $6.000 para el dueno (40%), sin intervencion manual.

HU-14/15/16 — Dueno registra un gasto y consulta la utilidad neta



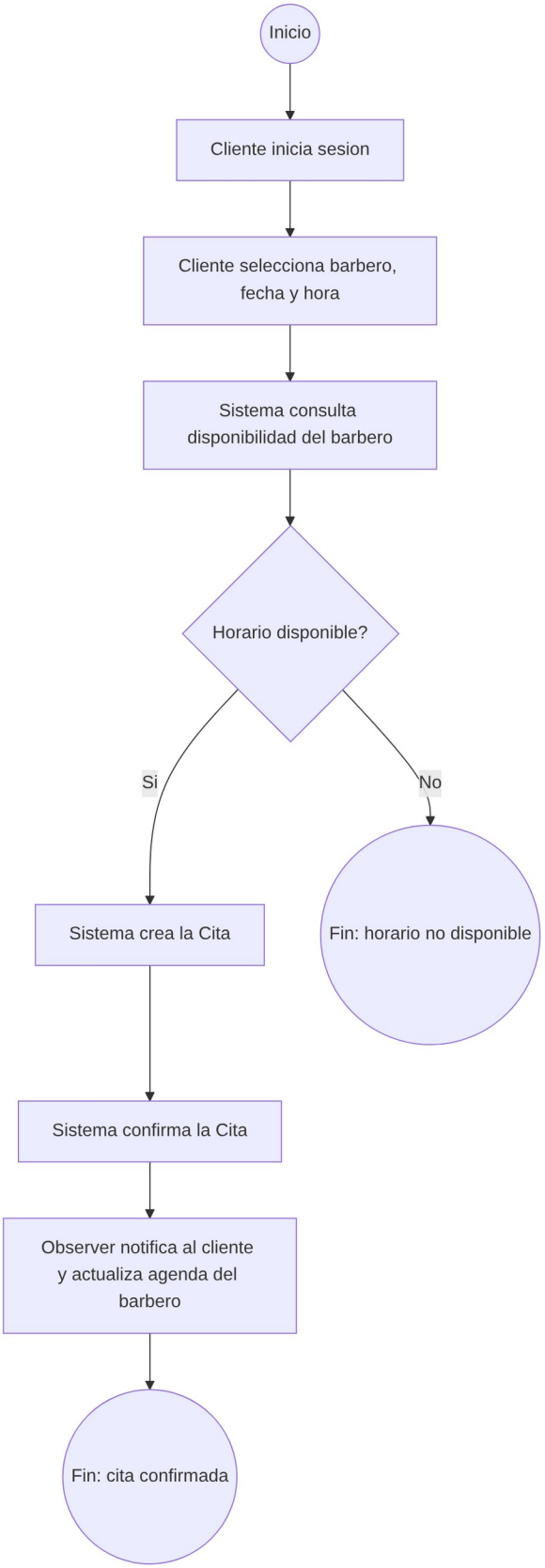


El dueno registra un gasto operativo ($500.000, Arriendo) y consulta el reporte del periodo: ingresos brutos $15.000, comisiones pagadas a barberos $9.000, gastos $500.000 y utilidad neta -$494.000 — calculado por ReporteFinanciero.Builder aplicando RN-03.

5. Diagramas de actividad del MVP

Un diagrama de actividad por cada grupo de historias de usuario implementadas, mostrando unicamente el flujo principal.

Historia de Usuario: HU-01 / HU-02 / HU-07 / HU-08 — Agendar cita

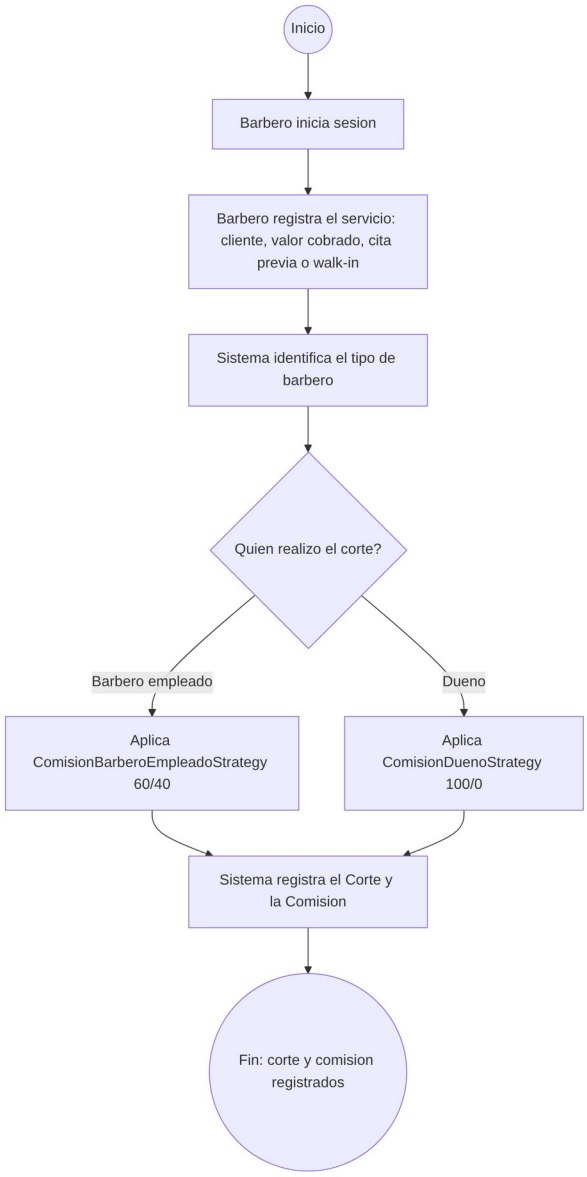


Enlace editable:

Abrir diagrama de actividad — HU-01 / HU-02 / HU-07 / HU-08 — Agendar cita

El cliente selecciona barbero, fecha y hora; el sistema consulta disponibilidad (HU-02) y valida que no haya cruce de horario (HU-07, RN-04) antes de confirmar la cita y notificar via el patron Observer (HU-08).

Historia de Usuario: HU-12 / HU-13 — Registrar corte y calcular comision

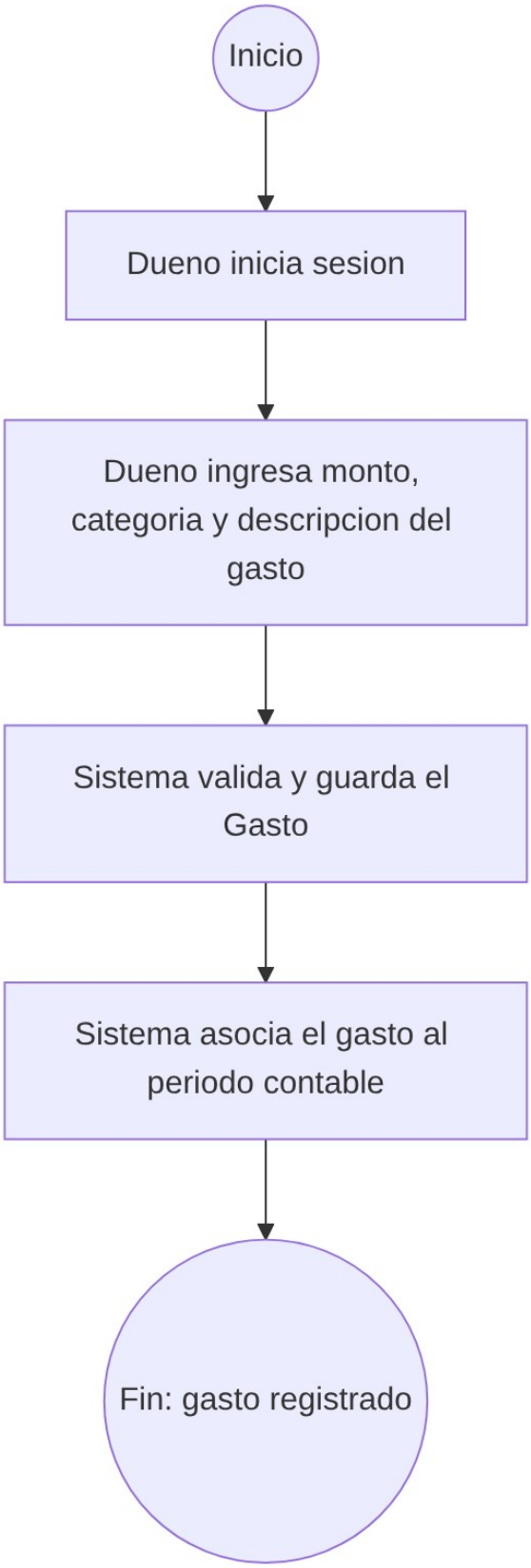


Enlace editable:

Abrir diagrama de actividad — HU-12 / HU-13 — Registrar corte y calcular comision

El barbero registra el servicio prestado; el sistema identifica si quien lo realizo es un barbero-empleado o el dueno y aplica la estrategia de comision correspondiente (patron Strategy).

Historia de Usuario: HU-14 — Registrar gasto operativo

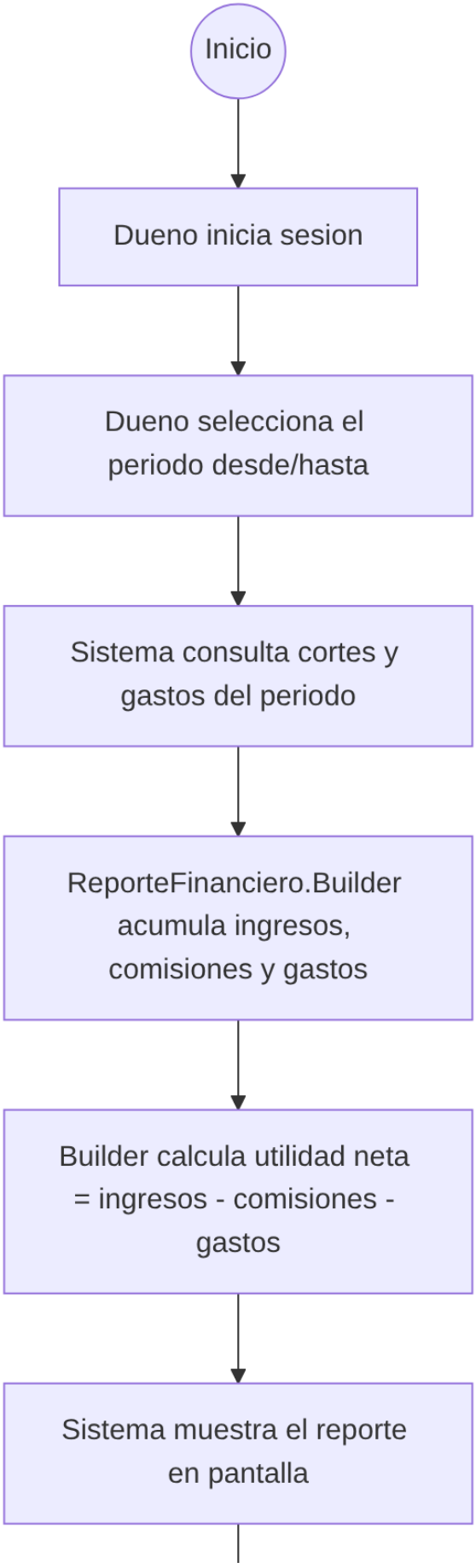


Enlace editable:

Abrir diagrama de actividad — HU-14 — Registrar gasto operativo

El dueno ingresa monto, categoria y descripcion del gasto; el sistema lo valida, lo guarda y lo asocia al periodo contable correspondiente.

Historia de Usuario: HU-15 / HU-16 / HU-20 — Ingresos, utilidad neta y exportacion

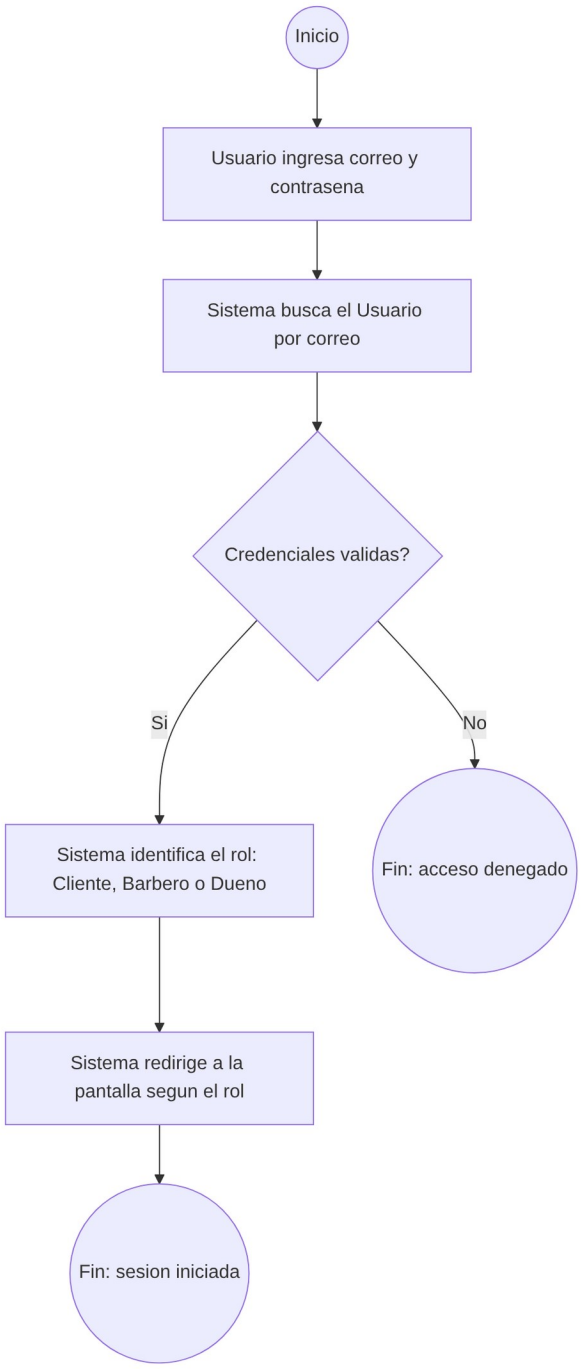

Inicio
Dueno inicia sesion
Dueno selecciona el periodo desde/hasta
Sistema consulta cortes y gastos del periodo
ReporteFinanciero.Builder acumula ingresos, comisiones y gastos
Builder calcula utilidad neta = ingresos - comisiones - gastos
Sistema muestra el reporte en pantalla

Enlace editable:

Abrir diagrama de actividad — HU-15 / HU-16 / HU-20 — Ingresos, utilidad neta y exportacion

El dueno selecciona un periodo; el sistema consulta cortes y gastos, y el patron Builder acumula los valores y calcula la utilidad neta (RN-03) antes de mostrar el reporte.

Historia de Usuario: HU-22 / HU-23 / HU-24 / HU-26 — Iniciar sesion por rol



Enlace editable:

Abrir diagrama de actividad — HU-22 / HU-23 / HU-24 / HU-26 — Iniciar sesion por rol

Un unico flujo de autenticacion (RN-06) valida las credenciales y redirige a la pantalla correspondiente segun el rol del usuario (Cliente, Barbero o Dueno), cumpliendo RN-07 de alcance de datos por rol.

6. Aplicacion de principios SOLID

El analisis completo (que estaba en E1, como se aplica en E2 y la evidencia concreta en codigo para cada uno de los 5 principios) esta diligenciado en la plantilla anexa (ver Anexos). Resumen:

- S — Responsabilidad Unica: Corte ya no calcula su propia comision; delega en CalculadoraComisionStrategy. Los *Service orquestan, no calculan.
- O — Abierto/Cerrado: una nueva politica de comision o un nuevo tipo de notificacion se agregan como una clase nueva (Strategy/Observer), sin modificar Corte, Barbero ni Cita.
- L — Sustitucion de Liskov: Dueno extiende Barbero y puede usarse en cualquier lugar donde se espera un Barbero (agendar cita, registrar corte) sin romper el contrato de la clase base.
- I — Segregacion de Interfaces: CalculadoraComisionStrategy, CitaSubject y ObservadorCita son interfaces pequenas, de una sola responsabilidad.
- D — Inversion de Dependencias: Barbero y Cita dependen de abstracciones (CalculadoraComisionStrategy, ObservadorCita), no de implementaciones concretas; los Service reciben sus dependencias por inyeccion de constructor (Spring).

7. Anexos

Repositorio de codigo

github.com/Estebanx999/Barberia

Incluye backend (Spring Boot), frontend (React) y esta misma carpeta docs/ con los diagramas y plantillas.

Prototipo Figma (Entregable 1, base del prototipo funcional)

Prototipo de alta fidelidad en Figma

Tablero de historias de usuario (Jira)

Proyecto DSILW en Jira

Enlaces editables de los diagramas

- C4 Nivel 1 — Contexto: ver seccion 2
- C4 Nivel 2 — Contenedores: ver seccion 2
- C4 Nivel 3 — Componentes: ver seccion 2
- C4 Nivel 4 — Codigo/Clases: ver seccion 2
- Diagramas de actividad (5): ver seccion 5

Plantillas diligenciadas

- Plantilla_GoF_Barberia.docx — analisis y seleccion de los 3 patrones GoF
- Plantilla_SOLID_Barberia.docx — analisis de los 5 principios SOLID

(Archivos incluidos junto a este documento y en docs/ del repositorio)

Bibliografia y referencias

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Martin, R. C. (2003). Agile Software Development: Principles, Patterns, and Practices. Prentice Hall. (Principios SOLID)
- Brown, S., & C4 Model contributors. (2024). The C4 model for visualising software architecture. https://c4model.com
- Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/
- React Documentation. https://react.dev
- PostgreSQL Documentation. https://www.postgresql.org/docs/